

---

# *Olyxee Document Integrity (ODI)*

---

## Neo Core Research & Engineering Task

Role: AI Engineer + Data Engineer

*Objective: (Build the first Integrity Runtime for AI Systems)*

### 1. What You Are Actually Building

ODI is NOT:

- an OCR system
- a document parser
- a normal extraction pipeline

ODI is:

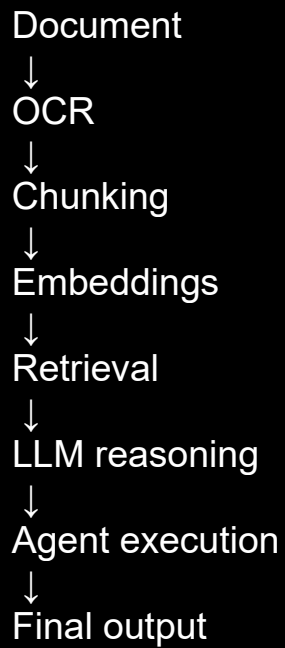
| an integrity infrastructure layer for AI systems.

The system exists to answer one question:

Did the AI preserve truth and constraints throughout the pipeline?

### 2. Why This Matters

Modern AI systems process information through many stages:



At each stage:

- information can degrade
- relationships can break
- numbers can mutate
- meaning can drift
- constraints can disappear

The dangerous part:

- outputs still look correct
- confidence scores remain high
- corruption becomes silent

ODI exists to detect this.

### **3. Research Inspiration**

This project direction is inspired partly by emerging research around:

- AI pipeline corruption
- semantic degradation
- long-horizon reasoning failures
- integrity loss in multi-step systems

Reference paper:

[arXiv Paper — 2604.15597](#)

Neo should study:

- silent corruption
- reasoning degradation
- information drift
- multi-step AI failure propagation

This paper is NOT the product.

It validates that the problem is real.

ODI is the engineering system built around solving it.

#### **4. ODI Core Idea**

Traditional systems ask:

What output did the AI produce?

ODI asks:

Can the output still be trusted after multiple transformations?

That is the core difference.

#### **5. Initial ODI Focus**

ODI V1 should focus on:

##### **Constraint-Preserving AI Pipelines**

Meaning:

- important constraints must survive AI transformations

Examples:

- totals remain mathematically correct
- VAT remains valid
- IDs remain linked

- dates remain consistent
- relationships remain intact
- structured meaning survives summarization

## **6. Sprint 1 Goal**

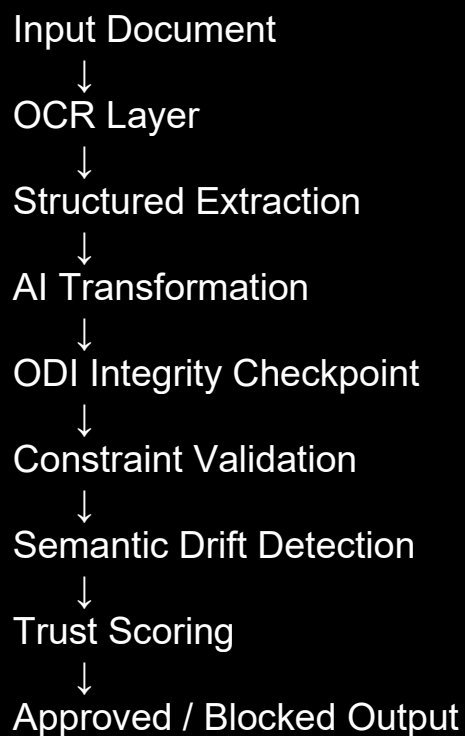
Build the first ODI Runtime prototype.

The purpose of Sprint 1 is NOT scale.

The purpose is proving:

| integrity validation between AI transformation stages.

## **7. Sprint 1 Architecture**



## **8. Deliverables + Completion Conditions**

### **Deliverable 1 Architecture Design**

#### **Task**

Design:

- system architecture
- service boundaries
- data flow
- integrity checkpoints
- validation pipeline

#### **Expected Output**

Neo should produce:

- architecture diagram
- service interaction flow
- folder structure
- pipeline explanation document

#### **How You Know It Is Complete**

This stage is complete when:

- another engineer can understand the system without explanation
- all pipeline stages are clearly defined
- data flow is visible end-to-end

## Deliverable 2 OCR Pipeline

### Task

Build:

- document upload system
- OCR extraction pipeline
- text extraction service

Supported:

- PDF
- PNG/JPG

### Expected Output

Example:

```
{
  "document_id": "doc_001",
  "pages": [
    {
      "page": 1,
      "text": "Invoice Total: R15,000"
    }
  ]
}
```

### How You Know It Is Complete

This stage is complete when:

- uploaded documents can be processed automatically
- text is extracted successfully
- page structure is preserved

### **Deliverable 3 Structured Extraction Layer**

#### **Task**

Extract:

- totals
- dates
- IDs
- entities
- invoice values
- structured fields

#### **Expected Output**

```
{  
  "invoice_total": 15000,  
  "vat": 1956.52,  
  "supplier": "ABC Logistics"  
}
```

#### **How You Know It Is Complete**

This stage is complete when:

- raw OCR text becomes structured JSON
- fields can be accessed programmatically

- extraction works consistently on sample documents

## **Deliverable 4 Constraint Validation Engine V1**

### **Task**

Build validation checks for:

- required fields
- mathematical consistency
- empty values
- invalid formats
- relationship consistency

Example:

- VAT calculations
- totals balancing
- matching IDs

### **Expected Output**

```
{
  "validation": {
    "passed": false,
    "issues": [
      "VAT mismatch detected"
    ]
  }
}
```

### **How You Know It Is Complete**

This stage is complete when:

- the system can automatically detect invalid outputs
- corrupted totals are flagged
- invalid relationships are detected

## **Deliverable 5 Semantic Drift Detection V1**

### **Task**

Research and prototype methods to detect:

- meaning mutation
- structural corruption
- missing constraints
- entity drift
- transformation degradation

Example:

Original:

Total = R15,000

Corrupted:

Total = R1,500

ODI should detect:

- unexpected transformation
- integrity degradation

### Expected Output

```
{  
  "drift_detected": true,  
  "severity": "high"  
}
```

### How You Know It Is Complete

This stage is complete when:

- ODI can identify at least some corrupted transformations
- drift signals become measurable
- integrity degradation is detectable

## Deliverable 6 Trust Scoring Layer

### Task

Build a trust scoring system based on:

- OCR confidence
- validation success
- drift detection
- missing fields
- integrity failures

## Expected Output

```
{
  "trust_score": 0.82,
  "risk_level": "medium"
}
```

## How You Know It Is Complete

This stage is complete when:

- every pipeline output receives a measurable trust score
- risky outputs can be separated from trustworthy outputs

## Deliverable 7 ODI Runtime Layer

### Task

Build middleware that:

- intercepts AI outputs
- validates integrity
- logs failures
- tracks transformations
- generates integrity metadata

## Expected Output

```
{
  "pipeline_stage": "extraction",
  "trust_score": 0.71,
  "integrity_flags": [
```

```
"entity_drift",  
"numeric_inconsistency"  
]  
}
```

## How You Know It Is Complete

This stage is complete when:

- every transformation stage becomes observable
- ODI can explain WHY outputs are risky
- integrity metadata is attached to outputs

## 9. Technical Stack

### Backend

- Python
- FastAPI

### AI / ML

- PyTorch
- Transformers
- OCR systems

### Infrastructure

- PostgreSQL
- Redis

### APIs

- REST APIs

## **10. Final Engineering Principle**

The most important rule in ODI:

Extraction alone is not enough.

ODI must continuously ask:

Did truth survive the transformation?

That is the entire purpose of the system.